FLINK FORWARD #ASIA 2021 ONLINE

实时即未来 REAL-TIME IS THE FUTURE

Flink引擎在快手的深度优化与生产实践

刘建刚 / 技术专家

目录

#1 快手Flink的历史及现状

#2 容错能力提升

#3 Flink引擎控制与实践

#4 快手批处理实践

#5 未来规划

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 快手Flink的历史及现状

发展历程

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE





FLINK FORWARD #ASIA 2021 ONLINE
实时即未来
REAL-TIME IS THE FUTURE

应用规模

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

50w
CPU核

2000+
作业数

6亿/秒
峰值处理

31.7万亿
日处理条数

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#2 容错能力提升

容错能力综述

#1 单点恢复

支持任意多个task fail原地恢复，确保long-running作业永不断流。

#2 跨集群容灾

双集群冷备、热备以及kafka双集群集成，极端情况下数据不丢失。

#3 黑名单

黑名单，涵盖多种场景的拉黑策略。

单点恢复

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

背景

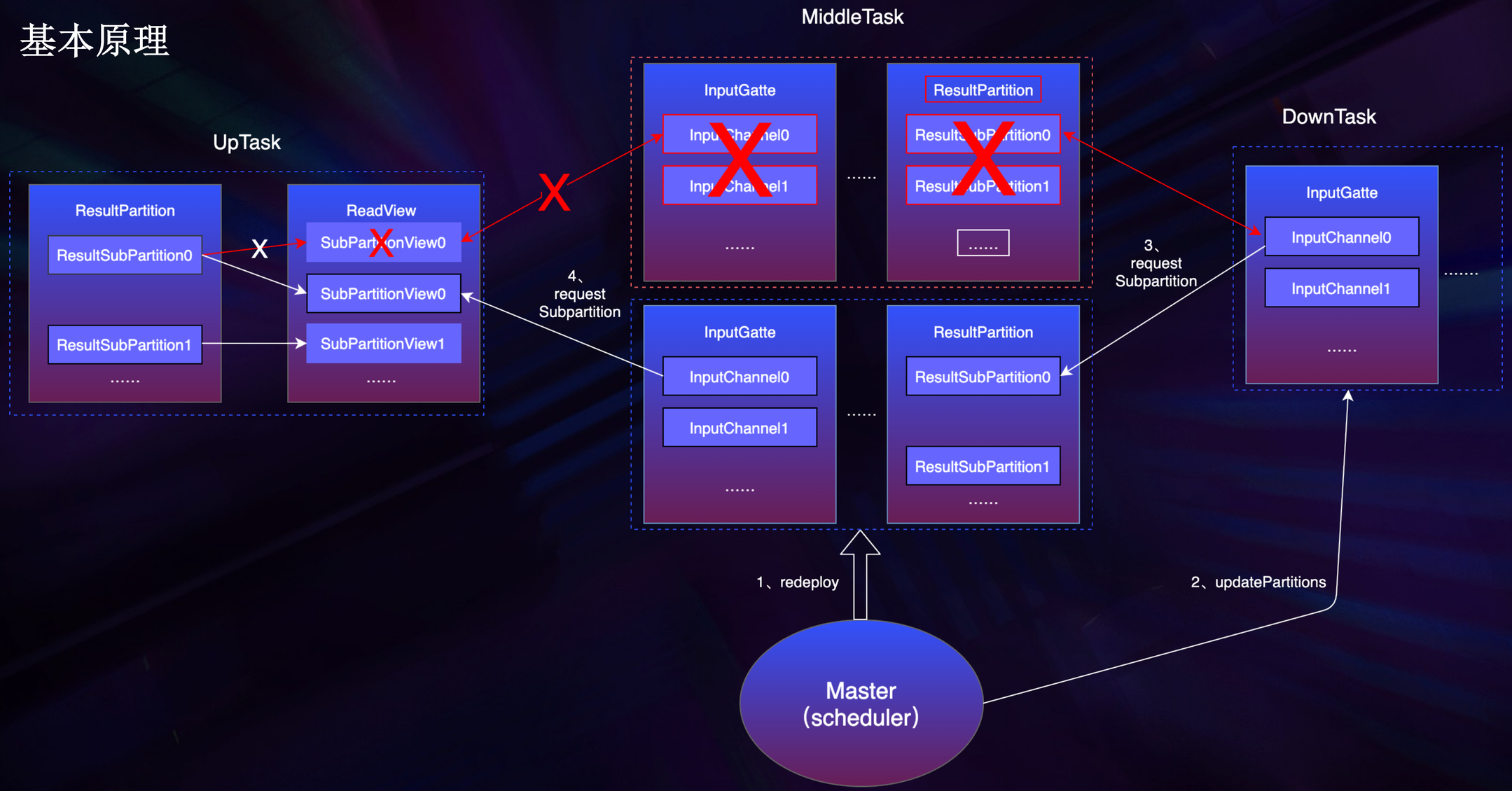
Flink为了做到exactly-once，一旦task出现故障必须全局重启。针对at-least-once和at-most-once的场景，这种重启策略带来的停顿时长有时是不可接受的，比如：

- 推荐等实时场景，对服务的可用性要求特别高。
- 模型训练等初始化很慢的应用，重启时间比较长。

针对以上场景，全局重启代价比较大，用户更希望能够做到单点恢复，即只恢复失败的task而不影响整个作业。

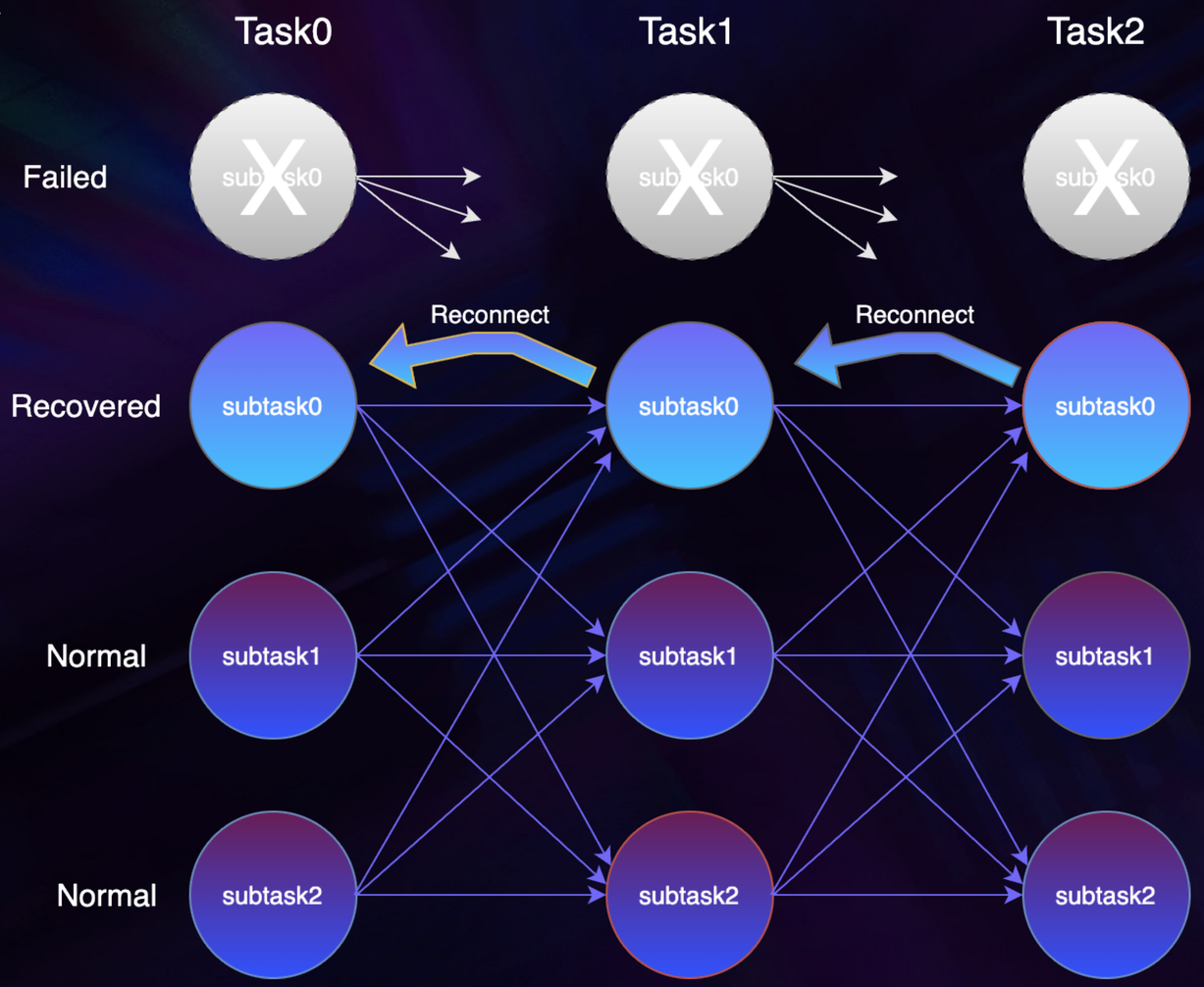
单点恢复

基本原理


MiddleTask
UpTask
DownTask
InputGatte
ResultPartition
InputChannel0
InputChannel1
ResultSubPartition0
ResultSubPartition1
ReadView
SubPartitionView0
SubPartitionView1
.....
3、request Subpartition
4、request Subpartition
1、redeploy
2、updatePartitions
Master (scheduler)

单点恢复

多Task恢复



跨集群容灾

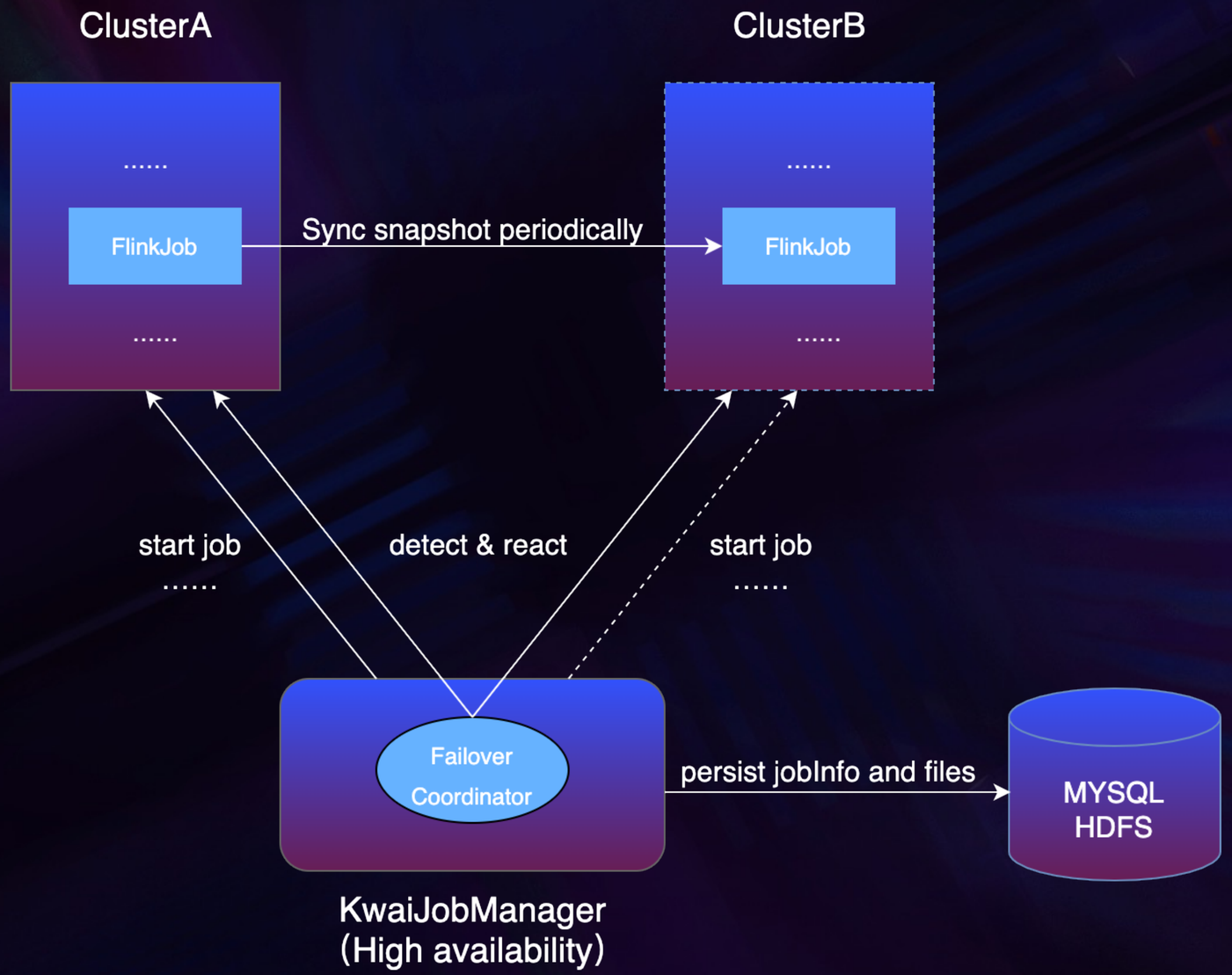
背景

集群故障会给集群上的服务、数据带来致命性的打击，即便是分钟级别的故障也会给公司带来巨大的损失，为此必须做到跨集群容灾。快手Flink具备以下三种集群跨集群应对方案：

- 冷备。作业在主集群启动，一旦出现故障立刻切换到备集群启动。
- 热备。作业在主备集群同时启动双跑。
- Kafka双集群集成。单集群Flink可以读写多集群的同名kafka topic并做到自动容灾。

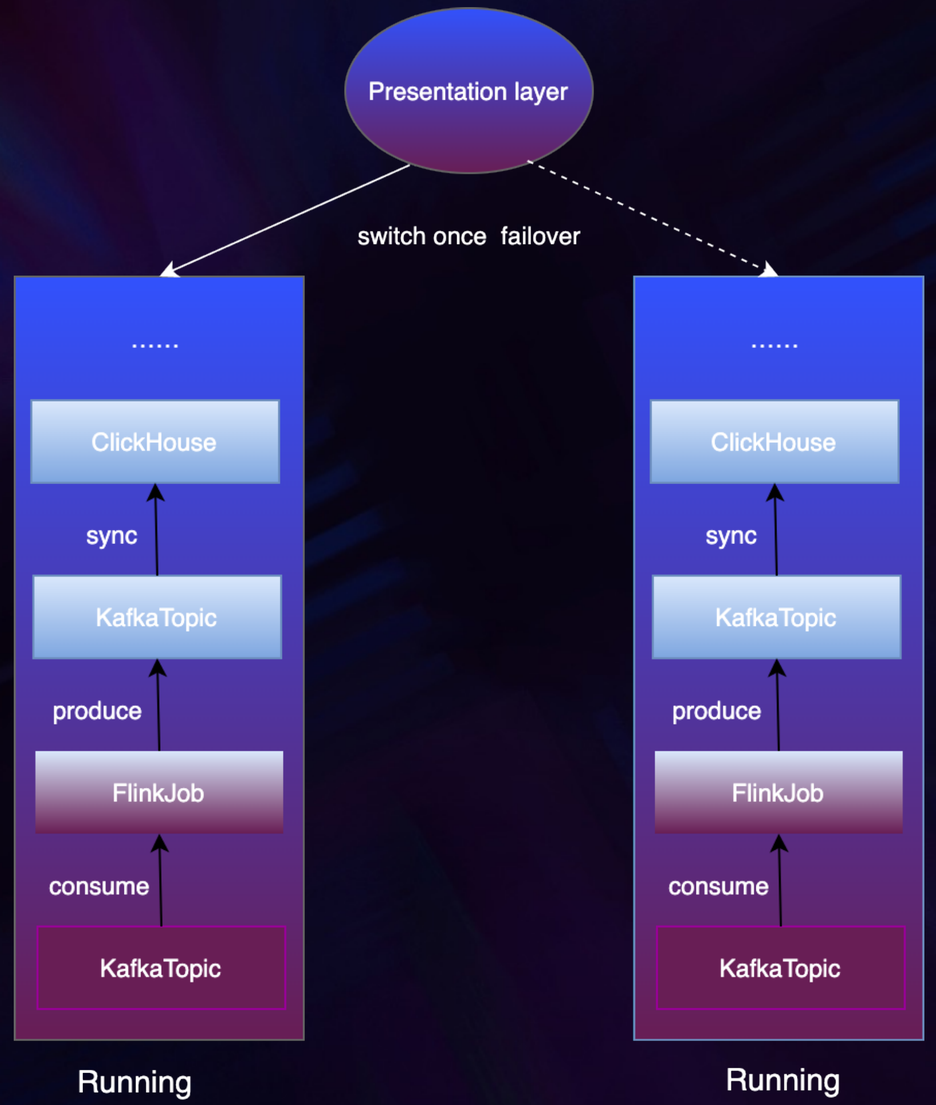
跨集群容灾

冷备


ClusterA
ClusterB
.....
FlinkJob
.....
Sync snapshot periodically
.....
FlinkJob
.....
start job
.....
detect & react
start job
.....
Failover
Coordinator
persist jobInfo and files
MYSQL
HDFS
KwaiJobManager
(High availability)

跨集群容灾

热备


Presentation layer
switch once failover
.....
ClickHouse
sync
KafkaTopic
produce
FlinkJob
consume
KafkaTopic
Running
.....
ClickHouse
sync
KafkaTopic
produce
FlinkJob
consume
KafkaTopic
Running

跨集群容灾

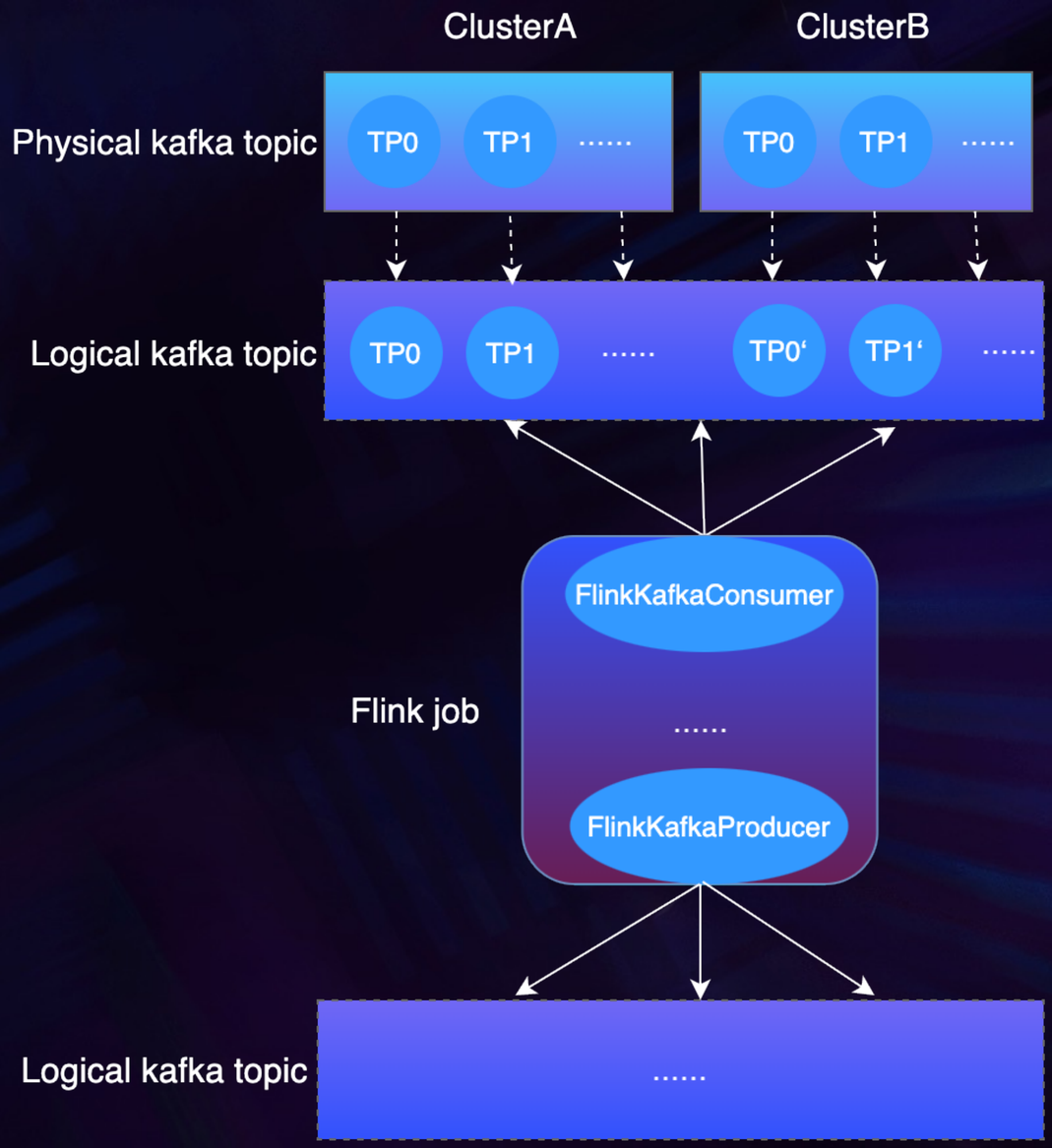
Kafka双集群集成

- 某个集群挂掉
  抽象成逻辑Topic的缩容，但是需要保存挂掉partition的state。

- Kafka单集群切双集群
  抽象成逻辑Topic的扩容。

- Kafka topic迁移
  抽象成逻辑Topic的缩容和扩容。

注：由于KwaiFlinkManager是高可用的，所以当前Flink集群挂掉，可以随时从另外一个Flink集群重新启动作业（比如startFromLatest），适用于无需数据保障的低优作业。



黑名单

问题

- 反复调度到故障机器上，导致作业频繁失败。
- 机器因为硬件或者网络等原因，导致Flink个别节点卡住但未失败。

策略

阈值拉黑

作业多次因为某一机器部署或者运行失败，超过配置阈值后，拉黑节点。

主动驱逐

建立异常分类机制，针对特殊异常，比如网络卡、磁盘故障，直接驱逐容器。

外部系统拉黑

暴露拉黑接口，联动外部系统，比如运维、yarn等，实现实时拉黑或重启。

快速容错总览

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

Task失败

- 单点恢复
- Local recovery
- 流程耗时优化

容器挂掉

- 资源预申请
- 冗余资源
- 黑名单

机器挂掉

- 快速发现与摘除
- 冗余资源

集群挂掉

- 冷备
- 热备
- Kafka双集群集成

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#3 Flink引擎控制与实践

引擎能力综述

#1 实时控制

在不停止作业的情况下调整作业行为。

#2 源端控制

数据源的控制能力，从根源上解决问题。

#3 均衡调度

包含task均衡调度和cpu均衡调度。

Flink实时控制

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

背景

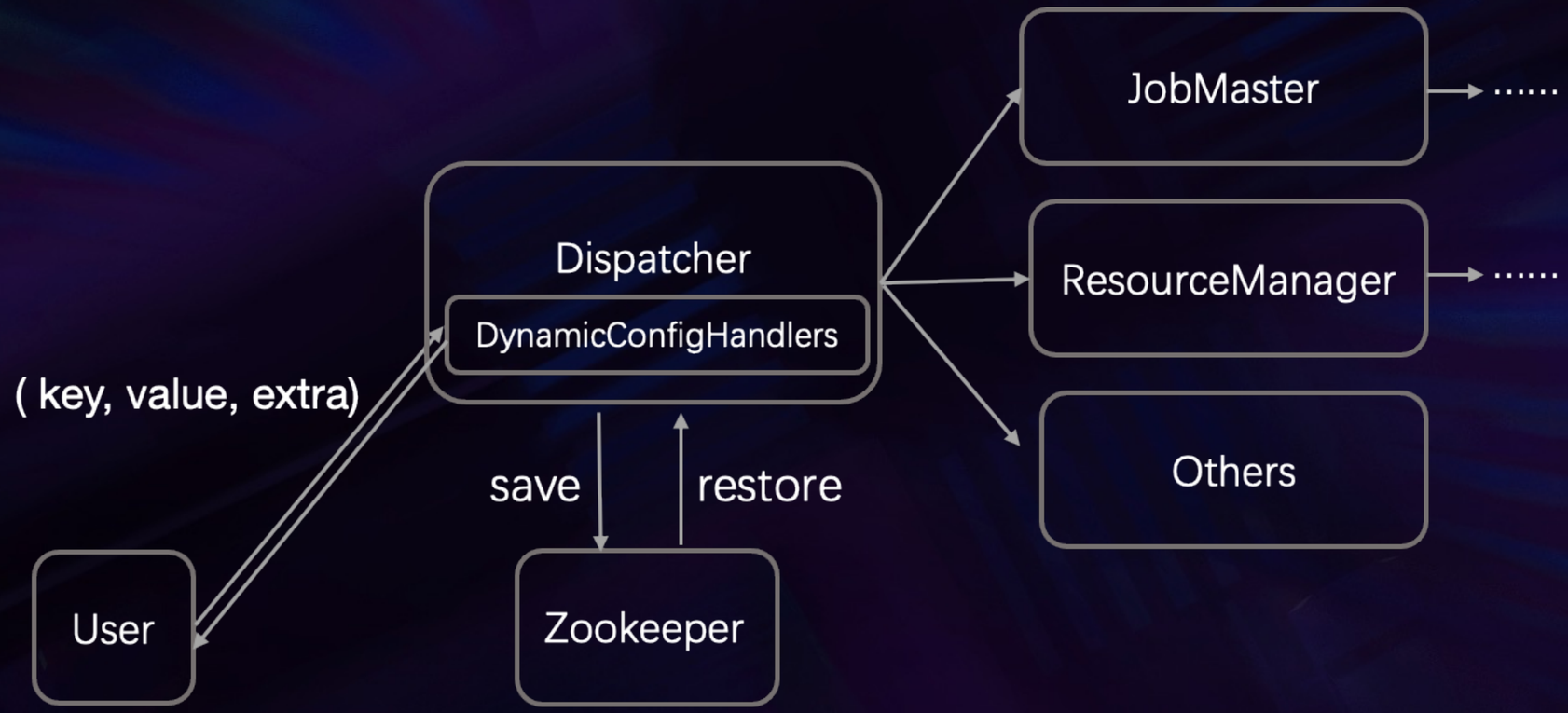
Flink作业都是long-running的实时计算，线上的作业调整时有发生，包括：

- 用户需要调整参数来改变作业逻辑，比如过滤条件；
- 系统需要降级、修改作业日志级别等运维手段。

任何的变更升级都需要重启作业，耗时从几分钟到几十分钟不等，这在活动、排查问题等关键时间点是无法接受的，如何做到实时控制至关重要。

Flink实时控制

将service和compute融合的一种轻量级的热更新模型，提供了交互式的控制模式，在不停止作业的情况下通过restful接口实时控制作业，应对各种极端情况及变更操作。



Flink实时控制

- 用户自定义动态参数，使用方法如下：
  - 继承某一RichFunction；
  - 通过getRuntimeContext().getDynamicConf()获取动态参数，并实现相应逻辑。

- 系统提供了丰富的实时控制能力：
  - 数据源限速
  - 数据源采样
  - 重置kafka offset
  - 开关快照
  - 更改日志级别
  - 拉黑节点

源端控制能力

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

背景

Flink在处理历史任务或者出现性能问题时，无法实时的处理消息事件，会引发以下问题：

- 各个Task处理进度不一致，会加重数据乱序、丢数据、对齐慢等问题。
- 快照持续变大，严重影响作业正常的数据处理。
- 流量和资源不可控。Flink会尽最大努力去消费数据，会引发cpu打满、oom加重等稳定性问题。

由于Flink是一种pipeline的实时计算，有效的数据源控制可以从根本上解决问题。

源端控制能力

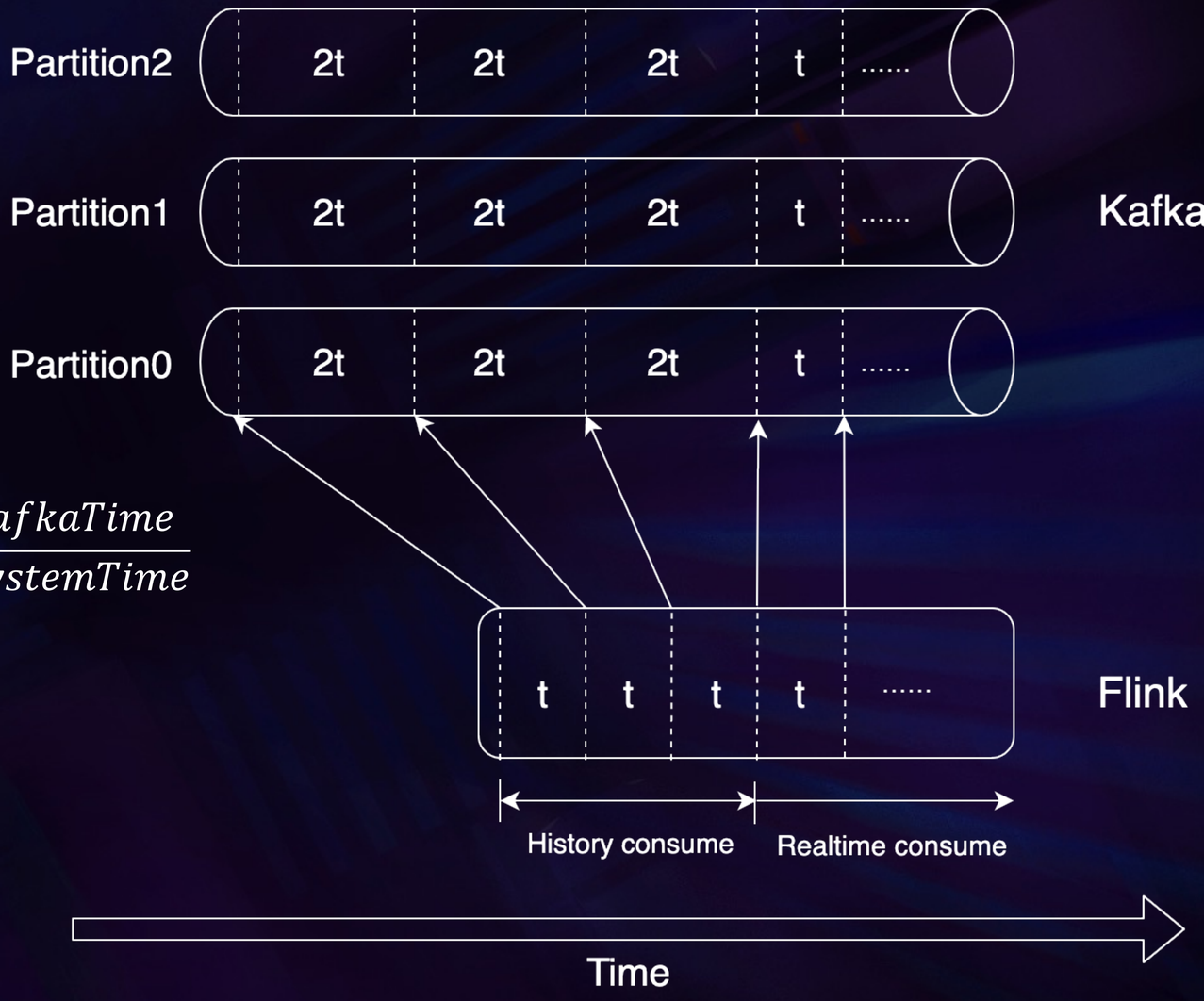
历史数据精准回放

功能特性

- 基于kafka时间戳等进行历史数据回放
- 统一历史消费和实时消费
- 解决了复杂大任务的稳定性问题

基本原理

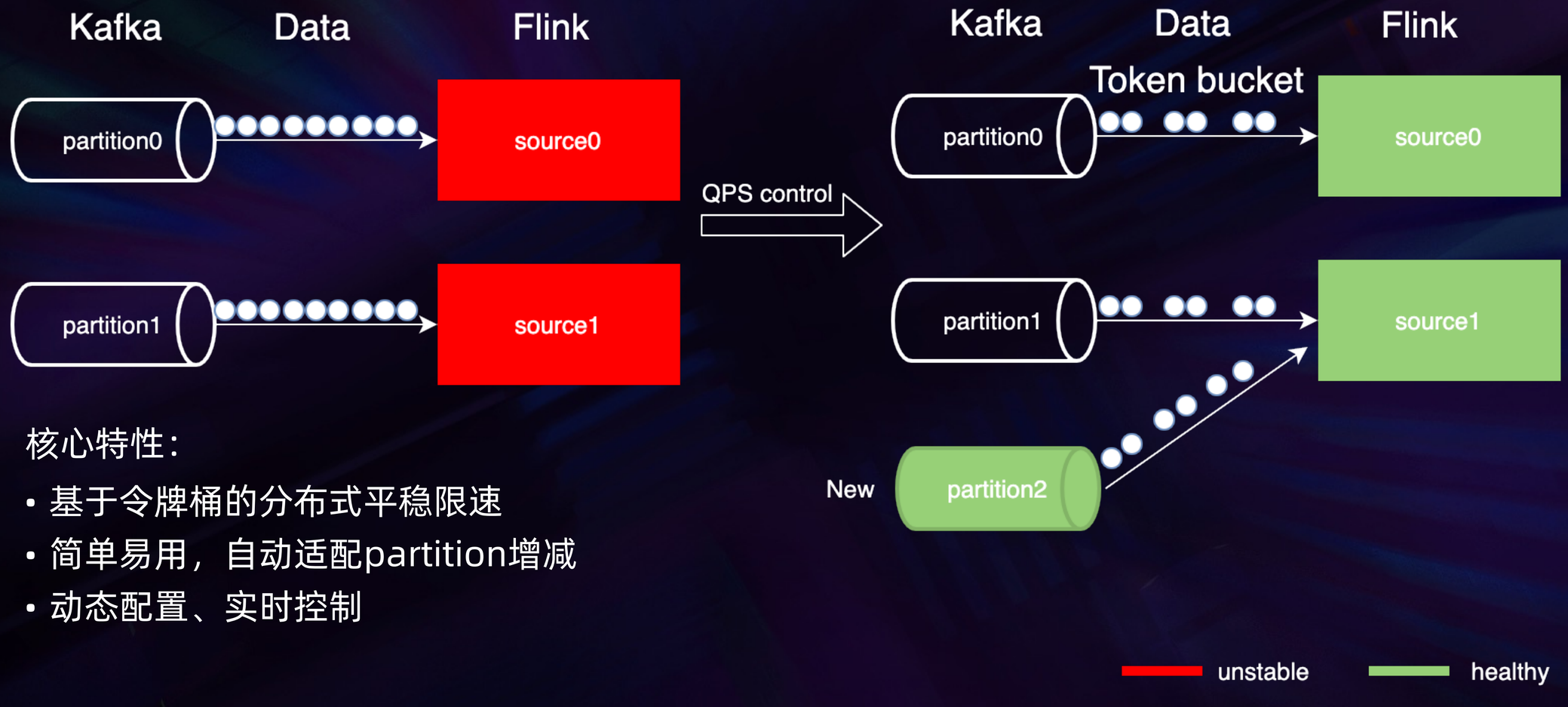
$$consumeRatio = \frac{currentKafkaTime - baseKafkaTime}{currentSystemTime - baseSystemTime}$$



源端控制能力

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

QPS限速



核心特性：

- 基于令牌桶的分布式平稳限速
- 简单易用，自动适配partition增减
- 动态配置、实时控制

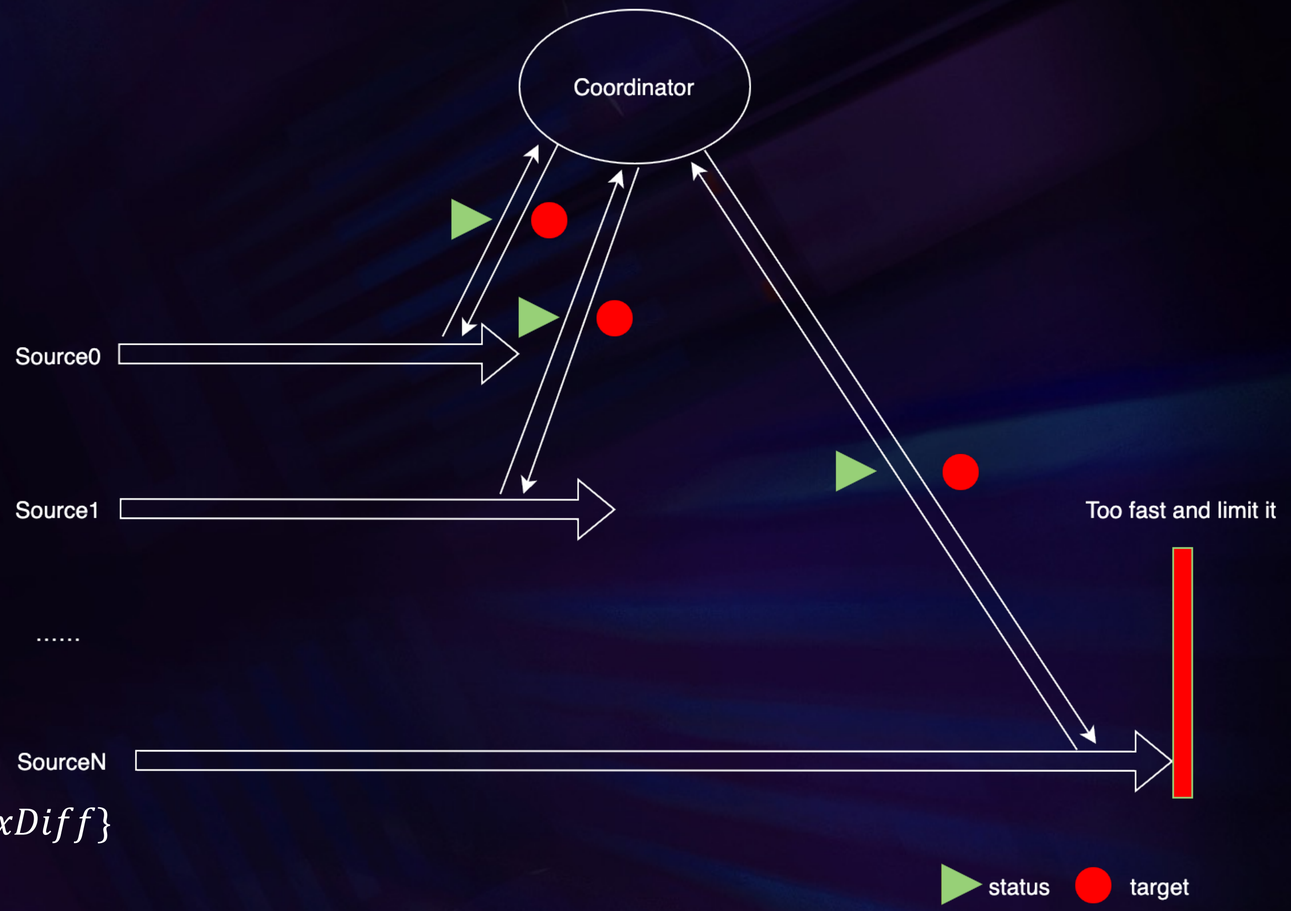
源端控制能力

数据源对齐

1. source的subtask定期汇报自己的watermark status（watermark、watermarkSpeed）
2. 中心节点计算并返回下一周期的target（所有source预期watermark的最小值）
3. source确保下一周期watermark不超过target

计算公式

$$target = min\{watermark + watermarkSpeed * interval + maxDiff\}$$



作业均衡调度

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

背景

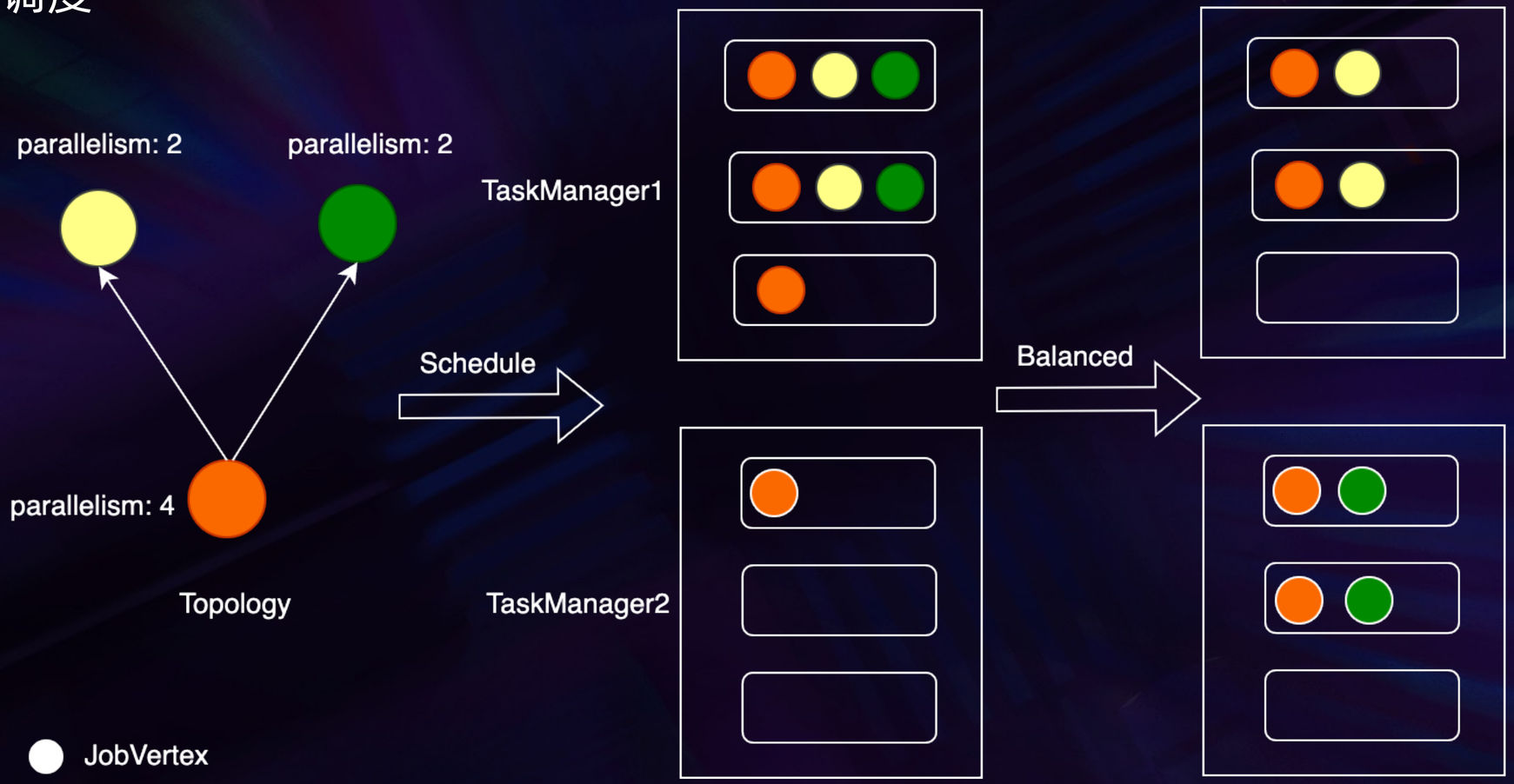
生产环境中，经常会出现资源使用不均衡的现象，如下所示：

- 作业task分布不均衡导致TaskManager资源使用不一致，作业性能瓶颈受限于最繁忙的节点。
- 作业task cpu使用不均衡，导致有些机器打满、有些机器很闲。

针对以上场景，我们开发了作业均衡调度和cpu均衡调度的功能。

作业均衡调度

Task均衡调度

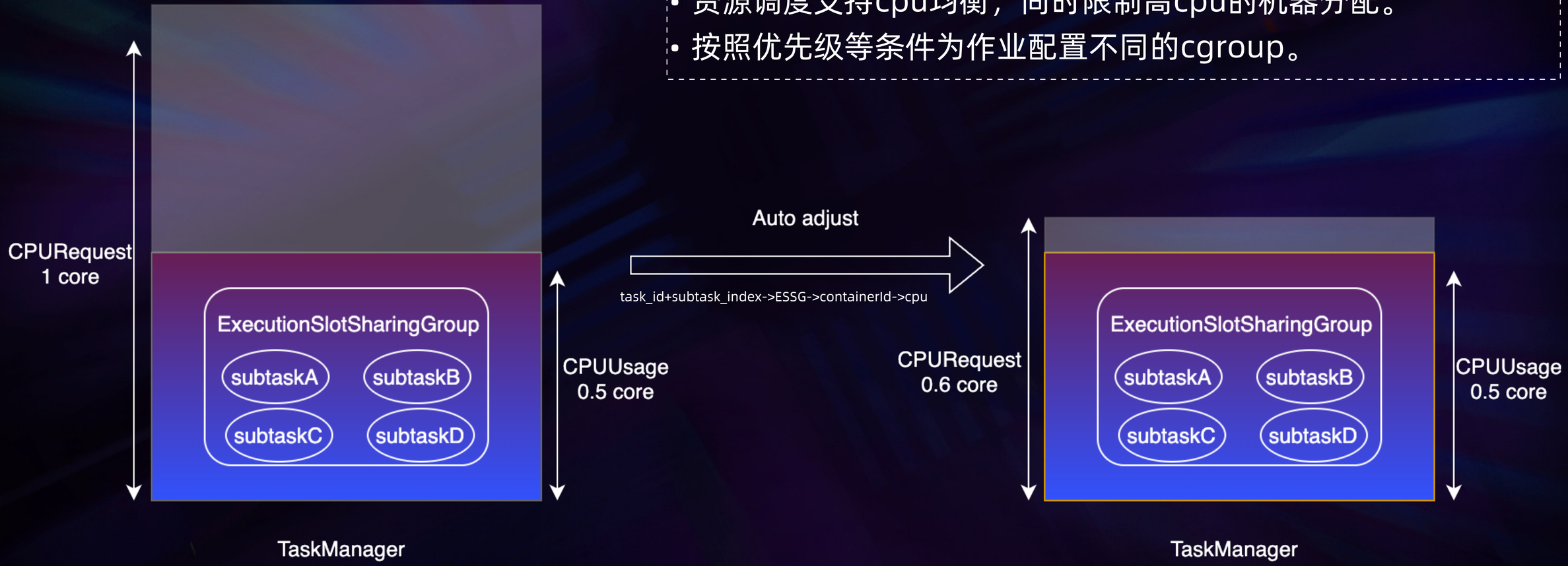


作业均衡调度

CPU均衡

关键点：

- 作业资源精准画像，根据映射关系分配资源（如图）。
- 资源调度支持cpu均衡，同时限制高cpu的机器分配。
- 按照优先级等条件为作业配置不同的cgroup。



FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#4 快手批处理实践

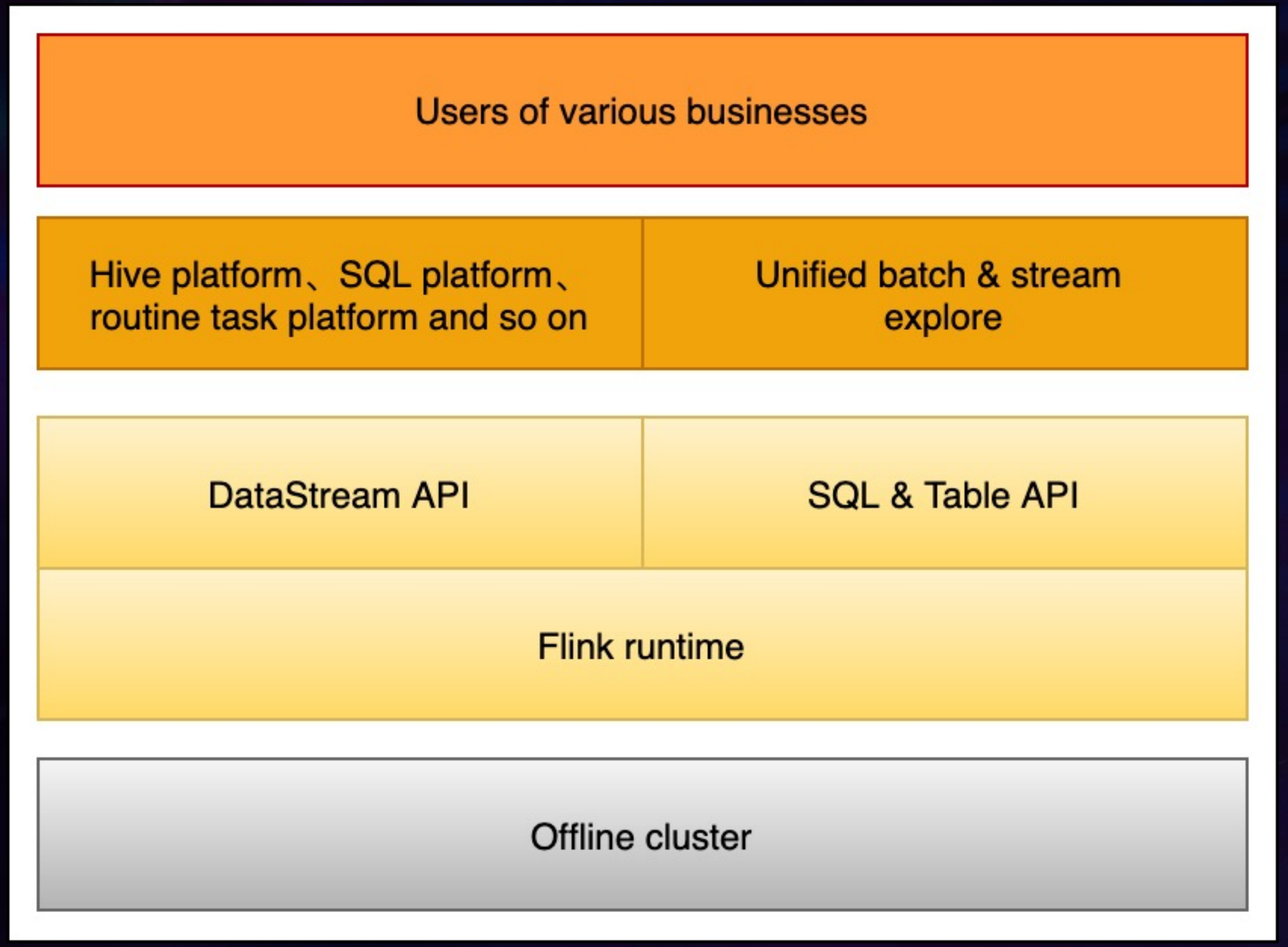
批处理综述

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 应用介绍

#2 优化及改进

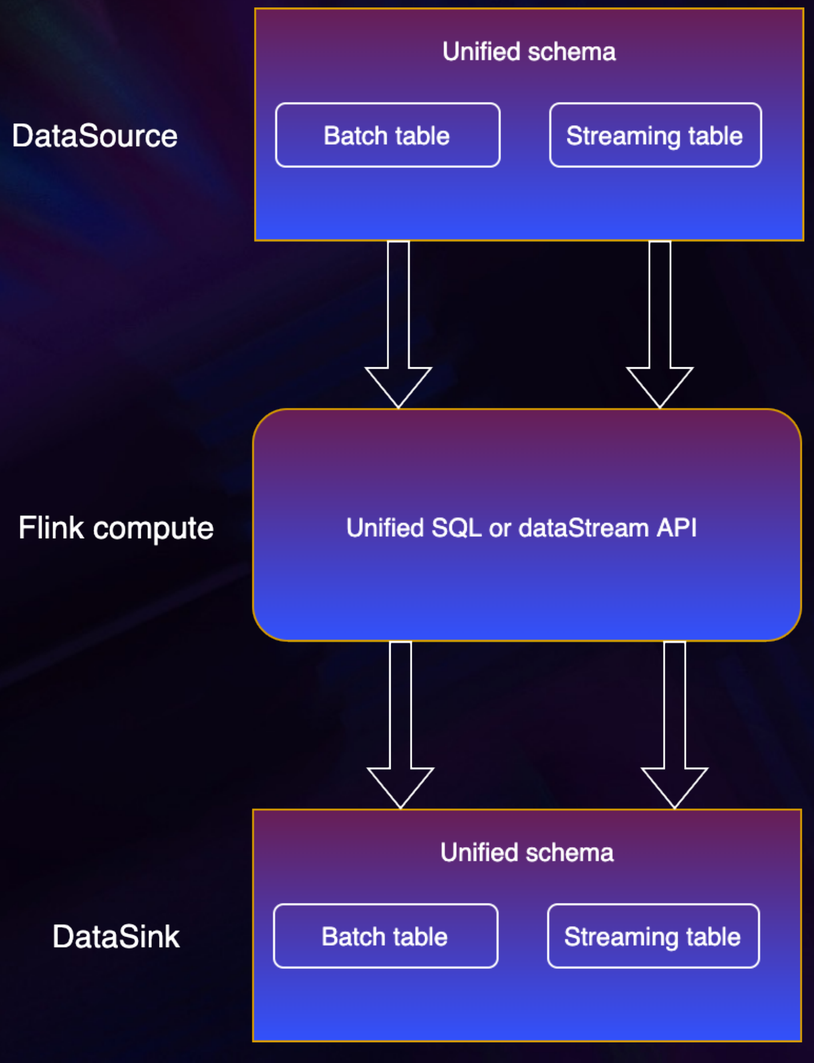
应用介绍


Users of various businesses
Hive platform、SQL platform、routine task platform and so on
Unified batch & stream explore
DataStream API
SQL & Table API
Flink runtime
Offline cluster

应用介绍

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

流批一体探索



应用场景：

- 指标计算
- 数据回溯
- 数仓加速

收益：

- 降低开发和运维成本
- 保持口径一致
- 节约资源

优化及改进



现状：上游失败，下游全部重算，消耗大量资源

改善：已经完成的下游不需要重新计算（默认开启）

优化及改进

优化项	做法	预期收益
Remote shuffle service	集成快手内部的shuffle service。	1、TaskManager挂掉不会导致partition lost，避免了数据丢失带来的重算。 2、利用shuffle service的文件聚合等功能，可以在磁盘IO、网络开销等方面带来显著收益。
动态资源调度	通过输入数据量来动态地决定Task的并发度。	合理利用资源，减少不必要的资源浪费和网络开销。
慢节点规避	1、自动识别并替代慢Task。 2、自动识别卡顿节点并规避。	避免长尾效应，减少总执行时间。
Hive UDF适配	解决部分Hive udf不兼容等问题。	Flink批处理和Hive的无缝对接。
部分SQL语法不支持	针对常规语法，Flink SQL进行补齐	Flink批处理和Hive的无缝对接。
优化Hive表partition split	增加缓存、多线程生成等方式	减少分片等待时间
支持更多压缩方式及配置	支持gzip、zstd等压缩配置	有效缓解不同场景下批处理大吞吐量下的压力

注：这里为优化项总括，部分已完成，部分正在做。

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#5 未来规划

未来规划

FLINK FORWARD #ASIA 2021 ONLINE 实时即未来 REAL-TIME IS THE FUTURE

#1 实时计算

继续增强Flink streaming的性能、稳定性、易用性。

#2 统一计算

统一快手的数据同步、转换和在离线计算。

#3 弹性可伸缩

在离线混部、弹性伸缩等。


FLINK FORWARD
#ASIA 2021
ONLINE
实时即未来
REAL-TIME IS THE FUTURE
THANKS